

无哈希函数的哈希表，以及如何充分利用你的随机比特

MIT CSAIL, kuszmaul@mit.edu。受 Fannie and John Hertz Fellowship 和 NSF GR

2026 年 7 月 15 日

摘要

本文考虑一个基本问题：一个存储 n 个 $(1 + \Theta(1)) \log n$ 比特键值对的哈希表能够提供多强的概率保证？过去关于这个问题的的工作一直受到已知哈希函数族局限性的制约：唯一能够实现低于 $1/2^{\text{poly} \log n}$ 失败概率的哈希表需要访问完全随机的哈希函数——如果使用已知的显式哈希函数族来实现相同的哈希表，其失败概率将变为 $1/\text{poly}(n)$ 。

为了绕过这些障碍，我们展示了如何构造一个具有与哈希表相同保证的随机化数据结构，但避免了直接使用哈希函数。在此基础上，我们能够构造一个使用 $O(n)$ 随机比特的哈希表，对于任意正常数 ϵ ，其失败概率达到 $1/n^{1-\epsilon}$ 。

事实上，我们证明这一保证甚至可以通过一个**简洁字典** (succinct dictionary) 来实现，即一个使用空间在信息论最优值的 $1 + o(1)$ 因子内的字典。

最后，我们还构造了一个简洁哈希表，其概率保证落在另一个极端，提供 $1/\text{poly}(n)$ 的失败概率，同时仅使用 $\tilde{O}(\log n)$ 随机比特。后一结果匹配了（在低阶项上）Dietzfelbinger 等人先前达到的保证，但具有更高的空间效率，并包含几个令人惊讶的技术组件。

1 1 引言

字典 (dictionary) 是任何支持对最多 n 个键的集合 S 进行插入、删除和查询的数据结构；字典通常还允许用户存储与每个键关联的值，这些值可以在查询时被检索。除非另有说明，我们假设机器字长为 $\Theta(\log n)$ 比特，这意味着键/值也是 $O(\log n)$ 比特。我们还隐式地要求字典最多使用线性空间（即 $O(n \log n)$ 比特），并且字典应该是**显式的** (explicit)（即可以在 $O(n)$ 时间内初始化）。事实上，本文中的字典具有更强的性质：它们可以在常数时间内初始化。

随机化字典通常也被称为**哈希表**¹。如果每个操作以至少 $1 - \delta$ 的概率在常数时间内完成，则称哈希表的**失败概率**为 δ ；如果 $\delta \leq 1/\text{poly } n$ ，则称其**以高概率成功**。

¹ 哈希表有时也被非正式地定义为使用哈希函数的字典问题的任何解决方案。我们有意对哈希表的定义采取更开放的视角，以便包含那些以与传统哈希表不同的手段实现相同目标的数据结构。

一个核心的开放问题是是否存在一个确定性的常数时间字典。这个方向上一个引人注目的成功是 Pătraşcu 和 Thorup 的**动态融合节点** (dynamic fusion node) [30]，它建立在 Fredman 和 Willard [15] 的早期工作之上，为非常小的键集合——即最多包含 $\text{polylog } n$ 个键、每个键 $\Theta(\log n)$ 比特的集合 S ——构造了一个确定性的常数时间字典。对于 $\Theta(n)$ 个键的集合，人们普遍认为（即使是非显式的）确定性常数时间字典是不可能的 [37]，但我们距离建立这一点的下界仍然非常遥远（参见 [8, 18, 27, 33, 36] 关于此问题的其他相关工作）。

在本文中，我们考虑这个问题的一个自然放松：哈希表能提供的最小失败概率是多少 [3, 16, 17]？我们提出了第一个实现显著亚多项式失败概率的哈希表。并且我们证明这样的哈希表甚至可以是**简洁的** (succinct)，即其使用的空间在信息论最优值的 $(1 + o(1))$ 因子内。

关于超高概率保证的过去工作。迄今为止，哈希表概率保证的研究与哈希函数族的研究 [7, 9, 11, 19, 21, 23–26, 29, 34, 35] 密切相关。如果能够访问完全随机的哈希函数，则已知如何实现显著亚多项式的失败概率 [3, 16, 17]。然而，正如 Goodrich、Hirschberg、Mitzenmacher 和 Thaler [17] 所观察到的，已知的用于模拟具有高独立性的常数时间哈希函数的技术 [11, 26, 35] 本身就是引入额外 $1/\text{poly}(n)$ 失败概率的随机化构造。减少这些失败概率的努力 [17] 仅能以 $\Omega(1)$ 的评估时间为代价来实现。

在已知的常数时间哈希函数族中，唯一被成功用于获得亚多项式失败概率哈希表的是**表格哈希** (tabulation hashing) [29]。虽然表格哈希的标准分析包含 $1/\text{poly}(n)$ 的失败概率，但 [29] 指出在某些参数范围内，真实的失败概率实际上是亚多项式的。确实，可以扩展 [29] 的技术来证明表格哈希函数以概率 $1 - 1/2^{\text{polylog } n}$ 实现负载均衡，从而允许构造一个失败概率为 $1/2^{\text{polylog } n}$ 的哈希表。据我们所知，这仍然是任何哈希表所达到的最小失败概率。

本文：具有近乎最优失败概率的哈希表。我们引入一个简单的数据结构，称为**放大旋转基数树** (amplified rotated trie)，对于我们可以任意选择的小正常数 ϵ ，它提供 $1/n^{1-\epsilon}$ 的失败概率。除非存在确定性常数时间字典，这几乎是人们能期望的最强保证：如果存在一个失败概率为 $1/n^{\epsilon}$ （对某个正常数 $\epsilon > 0$ ）的哈希表，那么这将意味着存在一个（非显式的）确定性常数时间字典。我们的结果显著改进了之前 $1/2^{\text{polylog } n}$ 的技术水平。

我们的第二个结果是，通过少量修改，同样的数据结构可以用来获得一个非常不同的保证。得到的哈希表称为**预算旋转基数树** (budget rotated trie)，使用 $\tilde{O}(\log n)$ 随机

比特，以关于 n 的高概率支持常数时间操作。这一保证（在之前 Dietzfelbinger 等人 [10] 使用更经典的基于哈希的技术的工作中也曾被实现）作为上述保证的自然对偶——不是试图在使用最多 $O(n)$ 随机比特的同时最小化失败概率，而是试图在维持标准 $1/\text{poly}(n)$ 失败概率的同时最小化随机比特。

预算旋转基数树的一个有趣特征是它们能够利用所谓的”**逐渐增加独立性的哈希函数**” (gradually-increasing-independence hash functions) [7, 23]。这些哈希函数最初由 Celis、Reingold、Segev 和 Wieder [7] 引入（随后由 Meka、Reingold、Rothblum 和 Rothblum [23] 使其更加高效），可以使用仅 $O(n \log \log n)$ 随机比特将 n 个球大致均匀地分布到 n 个桶中，但似乎有一个显著的缺点：它们需要 $\Theta((\log \log n)^2)$ 时间来评估。因此，过去将这些哈希函数应用于经典哈希表的工作 [32] 每次操作都需要 $\Omega(1)$ 的时间。我们的方法表明，这种逐渐增加独立性的哈希函数可能比之前认为的更广泛适用，并且可以用于常数时间数据结构的设计中。

实现简洁性。最后，我们将注意力转向空间效率。关于如何构造**简洁哈希表** (succinct hash table)（即一个将全空间 U 中的 n 个键/值对存储在

$$(1 + o(1))B(|U|, n)$$

比特中的哈希表，其中 $B(|U|, n) = \log \binom{|U|}{n}$ 是任何哈希表大小的信息论下界）也有大量工作（参见例如 [3, 4, 20, 31]）。

在第 6 节中，我们证明本文中的数据结构也可以在键/值为 $(1 + \Theta(1)) \log n$ 比特的参数范围内被做成简洁的。更一般地，我们给出一个黑盒变换，可以应用于任何字典，以获得一个概率保证几乎与原始字典相同的简洁字典。新字典使用 $B(|U|, n) + O(n(\log n)/\log \log n)$ 比特。

有趣的是，变换本身使用了我们的（非简洁的）预算旋转基数树作为关键算法组件。变换还使用了 Raman 和 Rao [31] 的一个归约，并且可以看作是 [31] 中给出的简洁字典（仅保证常数期望时间操作）的一个常数时间和随机高效的版本。

应用我们的变换，我们获得两个数据结构：一个使用 $O(\log n (\log \log n)^3) = \tilde{O}(\log n)$ 随机比特的简洁哈希表，同时以高概率支持常数时间操作；以及一个失败概率为 $1/n^{1-\epsilon}$ 的简洁哈希表，其中 ϵ 是我们可任意选择的小正常数。

绕过哈希函数瓶颈。我们结果的核心是一个简单但强大的观察：构造一个**不使用**哈希函数的哈希表是可能的，因此它不受困扰已知哈希函数构造的局限性的影响。特别地，我们从一个我们称为**旋转基数树** (rotated radix trie) 的简单随机化字典开始展开。与标准哈希表一样，旋转基数树使用线性空间并且是常数时间的（以高概率）。但与依赖哈希函数提供随机性的标准哈希表不同，旋转基数树使用直接嵌入到数据结构中的随机性。旋转基数树随后作为放大旋转基数树和预算旋转基数树的基础。

大纲。本文的其余部分安排如下。第 2 节介绍基本预备知识和约定——这些专门针对非简洁字典的设置，稍后在第 6 节讨论简洁字典时将建立略有不同的约定。第 3 节

介绍并分析旋转基数树。在此基础上，第 4 节给出一个失败概率达到 $1/n^{1-\epsilon}$ 的哈希表，第 5 节给出一个使用 $O(\log n \log \log n)$ 随机比特的高概率哈希表；在附录 A 中，我们证明后一保证也可以扩展到机器字长为 $\Theta(\log n)$ 比特的情况。最后，在第 6 节中，我们展示如何将任何线性空间哈希表转换为简洁哈希表，同时几乎保持数据结构的随机化保证。

2 2 预备知识和约定（针对非简洁字典）

我们现在提出几个讨论（非简洁）字典的预备定义和约定。这些约定在整篇论文中使用，除了第 6 节我们考虑简洁字典的地方。在讨论非简洁字典与简洁字典时，我们最终使用略微不同的约定，因为在非简洁设置中，有许多可以在不失一般性的情况下做出的标准简化（但这些简化在简洁设置中不成立）。

键、值和字典。令 $U = [\text{poly}(n)]$ 是所有可能 $(\log n)$ 比特键的集合，令 $V = [\text{poly}(n)]$ 是所有可能 $(\log n)$ 比特值的集合。一个字典是一个数据结构，它存储来自 U 的一个键集合，并将每个键 x 与值 $y \in V$ 关联起来。字典支持三种操作：Insert(x, y) 将键 x 添加到集合中（如果它尚不存在），并将相应的值设置为 y ；Delete(x) 移除 x ；Query(x) 报告键 x 是否存在，如果存在则返回相应的值。

在讨论非简洁字典时，我们（不失一般性地）关注**固定容量字典**，即允许一次最多有 n 个键的字典。这样的字典可以通过在字典大小变化常数因子时简单地重建字典（以去摊销的方式）来实现动态调整大小的字典。除非另有说明，我们将隐式地要求字典必须最多使用线性空间（即 $O(n \log n)$ 比特）并具有 $O(1)$ 的初始化时间。

简化字典的标准技术。有几个标准归约可以用来简化维护线性空间字典的问题。

我们可以不失一般性地假设字典的生命周期只有 $O(n)$ 个操作。确实，更长的操作序列可以被分解为大小为 $O(n)$ 的阶段，并且字典可以在每个阶段从头开始重建（即所有元素从一个字典实例逐步移动到另一个新的字典实例）。重建成本可以分摊到整个阶段，使得操作的渐近运行时间得以保持。²

² 就我们的目的而言，重建不会采样新的随机比特。一旦字典的随机比特被选定，它们将永久固定。

由于每个阶段的生命周期只有 $O(n)$ 个操作，我们可以用以下简单方法实现删除：简单地将元素标记为已删除，并将这些元素的实际移除推迟到下一次重建。因此，在设计用于实现每个阶段的字典时，我们可以不失一般性地假设执行的唯一操作是插入/查询。

因此，我们将在整篇论文中假设，每当我们讨论非简洁字典时，正在执行的操作序列长度为 $O(n)$ ，并且仅由插入/查询组成。

随机化。随机化字典可以访问随机比特流——字典可以在 $O(1)$ 时间内访问流中的下一个 $(\log n)$ 比特。在分析随机化字典时，目标是界定任何给定操作的失败概率。我们

强调，在这个语境中，失败不是指缺乏正确性，而是指缺乏及时性。每当一个操作需要超常数时间时，字典就失败了。（稍后在第 6 节中，当我们考虑简洁字典时，我们还将允许关于空间消耗的失败。）

我们所有的字典共享一个性质：一旦发生失败，其余所有操作（在当前阶段的 $O(n)$ 个插入/查询中）也会失败。我们不会显式地指定当失败发生时字典的行为，因为在那时，阶段中剩余的每个操作都可以花费线性时间。

我们注意到随机化数据结构是针对**健忘对手**（oblivious adversary）分析的，这意味着执行的插入/删除/查询序列是独立于字典使用的随机比特确定的。我们还注意到字典的失败概率是在每个操作的基础上确定的。例如，如果一个字典的失败概率为 p ，并用于 $1/p$ 个操作，那么发生一些失败是合理的。³

³ 此外，不同步骤之间（以及不同阶段之间）的失败可能是相关的。例如，如果我们使用 r 个随机比特，而一个对手猜中了它们，那么该对手可以以概率 $1/2^r$ 一直强制产生失败。

3 3 热身数据结构：旋转基数树

在本节中，我们提出一个简单的随机化常数时间字典，称为**旋转基数树**（rotated radix trie），它作为后续各节中数据结构的基础。

出发点： n 叉基数树。我们数据结构的出发点将是经典的 n 叉基数树。树的每个内部节点可以看作一个大小为 n 的数组，其中数组的第 j 个条目存储指向孩子 j 的指针（如果存在这样的孩子），否则存储一个空指针。树的叶子对应于数据结构中的键（也是我们存储值的地方）。一般来说，存在一个根到叶路径为 $j_1, j_2, j_3, \dots, j_d$ 的叶子当且仅当键 $j_1 \circ j_2 \circ j_3 \circ \dots \circ j_d \in [n^d] = [U]$ 存在。

使得 n 叉基数树成为一个有趣出发点的原因是，该树确定性地支持常数时间操作。但它不支持的是空间效率：可能有高达 $\Omega(n)$ 个内部节点，每个节点是一个大小为 n 的数组，合起来需要 $\Theta(n^2)$ 的空间来实现。

使用随机性节省空间：旋转基数树。现在我们以一种非常简单的方式向数据结构添加随机性。将树的内部节点标记为 $1, 2, \dots, m$ ($m \leq O(n)$)，并将用于实现每个内部节点 $i \in [m]$ 的数组称为 A_i 。当数据结构被初始化时，我们为每个内部节点 i 分配一个随机旋转量 r_i ，从 $\{0, 1, \dots, n-1\}$ 中均匀随机地选取。旋转量 r_i 作为节点 i 的一部分存储。

r_i 的目的是对数组 A_i 应用一个随机循环旋转。也就是说，如果一个指针本来会存储在 A_i 的位置 j 中，现在它改为存储在 A_i 的位置 $((j + r_i) \bmod n)$ 中。

最后，在对每个数组 A_i 旋转 r_i 之后，我们现在将数组 $A_1, A_2, \dots, A_m$ 叠加在一起，并将它们的所有内容存储在单个大小为 n 的数组 A 中。当然， A 的第 j 个位置可能负责存储来自多个 A_i 的元素。只要每个条目中存储的元素数量相对较小，这就没问题：我

们简单地将 A 的每个条目实现为一个**动态融合节点** (dynamic fusion node) [30], 这是一个确定性的常数时间线性空间字典, 能够一次存储最多 $\Delta = \text{polylog } n$ 个键/值对。

如果在将数组折叠为单个数组 A 之前, 旋转后的数组 A_i 的第 j 个位置存储了一个指向数组 $A_{i'}$ 的指针, 那么之后动态融合节点 $A[j]$ 存储键值对 $(i, (i', r_{i'}))$ 。在这个设置中, 我们将对 $(i', r_{i'})$ 称为指向 $A_{i'}$ 的**指针**, 因为它决定了我们指向的是哪个数组 $A_{i'}$ 以及在哪里找到 $A_{i'}$ 的条目。类似地, 如果在折叠数组之前, 旋转后的数组 A_i 的第 j 个位置直接存储了一个指向值 (而非另一个数组 $A_{i'}$) 的指针 p , 那么动态融合节点 $A[j]$ 存储键值对 (i, p) 。

分析旋转基数树。为了分析旋转基数树, 我们必须证明, 以关于 n 的高概率, A 的每个条目负责存储来自最多 $\Delta = \text{polylog } n$ 个不同 A_i 的条目。

让我们首先建立一些在本文其余部分有用的约定。在讨论基数树时, 我们将数组 $A_1, A_2, \dots, A_m$ 称为**节点** (或有时称为**内部节点**), 并将每个 A_i 的非空条目 (即包含指针的条目) 称为**球** (balls)。

总的来说, 树中有 $O(n)$ 个球。每个球 b 由一个对 $(s, c) \in [m] \times [n]$ 指定, 其中 $s \in [m]$ 是球的**源节点** (即包含该球的节点), $c \in [n]$ 是球的**孩子索引** (即 A_i 中 b 在逻辑上存储的索引)。随机旋转数组 A_i 然后叠加它们以获得单个数组 A 的效果是, 每个球 $b = (s, c)$ 被映射到 A 中的位置 $\phi(s, c) := c + r_s$ 。我们将 A 的条目称为**桶** (bins), 因此每个球 b 被映射到桶 $\phi(b)$ 。每个桶 $j \in [n]$ 的动态融合节点存储键值对 (b, p) 的集合, 其中 b 遍历满足 $\phi(b) = j$ 的球, p 是对应于球 b 的指针。

对于 $i \in [m]$ 和 $j \in [n]$, 令 $X_{i,j}$ 是指示节点 i 是否向桶 j 放入一个球的 0-1 随机变量。 $X_{i,j}$ 在桶 j 之间不是独立的, 但它们在节点 i 之间是独立的, 因为每个 $X_{i,j}$ 是随机比特 r_i 的函数。因此, 桶 j 中的球数 Y_j (由 $Y_j = \sum_{i=1}^m X_{i,j}$ 给出) 是独立指示随机变量之和。

$O(n)$ 个球中的每一个在桶 j 中的概率是 $1/n$, 所以 $\mathbb{E}[Y_j] = O(1)$ 。因此, 由 Chernoff 界可知, $Y_j \leq \text{polylog } n$ 以关于 n 的高概率成立。Chernoff 界实际上告诉我们 $Y_j \leq \text{polylog } n$ 以概率 $1/n^{\text{polylog } n}$ 成立, 因此我们甚至达到了略微亚多项式的失败概率。

组合各部分。如果按照第 2 节的方式实现删除, 我们得到以下结果:

命题 1。旋转基数树是一个随机化的线性空间字典, 可以一次存储最多 n 个 $(\log n)$ 比特的键/值, 并以概率 $1 - 1/n^{\text{polylog } n}$ 在常数时间内支持每个操作。

值得花一点时间说明如何初始化我们的数据结构。随机旋转量 r_i 可以延迟初始化, 使得 r_i 在节点 i 第一次使用时才生成。此外, 我们实际上不必支付初始化任何数组的成本, 因为我们可以使用标准技术在常数时间内模拟零初始化的数组 (参见 [1] 或 [5] 第 1.6 节的问题 9)。因此, 我们的旋转基数树可以在常数时间内初始化。

审视当前进展。旋转基数树本身并没有在我们关心的任何问题取得重大进展: (1) 实现超高概率保证, 以及 (2) 使用接近对数的随机比特数。我们实现了略微亚多项式的失败概率, 但远未达到我们的目标 $1/n^{1-\epsilon}$ 。

然而，使旋转基数树有用的是，随机性在数据结构中的角色非常**简单**。唯一的随机性来源是旋转偏移量 $r_1, r_2, \dots, r_m$ 。在这个意义上，旋转基数树偏离了设计常数时间字典的标准模式。数据结构中的随机性不是用于哈希元素，而是用于对稀疏数组应用随机旋转。

由于随机性的角色在后续各节中很重要，我们通过讨论随机性如何随时间被重新利用来结束本节。考虑数据结构在包含许多插入/删除的长时间段内如何演化。随着树的形状变化，每个数组 A_i 将被重新利用来表示树的不同部分。这意味着随机旋转量 r_i 与键空间交互的方式也随时间变化，同一个 r_i 在不同时间应用于树中的不同节点（从而应用于键空间的不同部分）。 r_i 的重新利用有一个有趣的后果：即使两个时间点 t_1 和 t_2 存储完全相同的键/值对集合 S ，旋转基数树的形状在两个时间之间也可能有显著差异。

4 4 放大旋转基数树

在本节中，我们修改旋转基数树，将其失败概率（即给定操作需要超常数时间的概率）降低到 $1/n^{1-\epsilon}$ ，其中 ϵ 是我们选择的一个正常数。我们将这个新数据结构称为**放大旋转基数树**（amplified rotated radix trie）。

在(非旋转的)树中存储溢出球。每当一个球 b 被插入到一个已经包含 $\Delta = \text{polylog } n$ 个其他球的桶 j 中时，球 b 被视为一个**溢出球**。由于每个桶是一个容量为 Δ 的动态融合节点，我们无法在桶中存储溢出球。

我们改为将溢出球存储在一个辅助数据结构 Q 中，该结构被实现为一个 n^δ 叉树，其中 $\delta > 0$ 是某个正常数。

辅助数据结构 Q 在常数时间内支持对溢出球的插入/查询。另一方面， Q 不是空间高效的。如果有 q 个溢出球，那么 Q 可能使用高达 qn^δ 的空间。为了确立我们的字典使用线性空间，我们必须证明

$$\Pr[q \geq n^{1-\epsilon}] \leq O\left(1/n^{1-\epsilon}\right). \quad (1)$$

问题：共享源节点的球之间的依赖关系。然而，我们当前的数据结构还不满足 (1)。这是因为，每当多个球共享同一个源节点时，它们的分配变得紧密关联。例如，假设旋转基数树 R 只有 2Δ 个内部节点，且每个内部节点 $i \in \{1, 2, \dots, 2\Delta\}$ 包含 (n/Δ) 个球 $(i, 1), (i, 2), \dots, (i, (n/\Delta))$ 。以概率 $1/n^{2\Delta} = 1/2^{\text{polylog } n}$ ，每个内部节点 $i \in \{1, 2, \dots, 2\Delta\}$ 有随机旋转量 $r_i = 0$ 。这导致桶 $1, 2, \dots, (n/\Delta)$ 各包含 2Δ 个球——换句话说，系统中一半的球是溢出球。这意味着

$$\Pr[q \geq \Omega(n)] \geq 1/2^{\text{polylog } n}.$$

也就是说，我们使用 Q 存储溢出球的失败概率并不比我们在第 3 节中没有 Q 时达到的失败概率更好。

减少依赖关系。使得上述病态例子成为可能的原因是，我们的旋转基数树中可能只有很少的内部节点。这使得只有很少的随机比特影响旋转基数树的结构，阻止我们实现任何超高概率保证。

为了解决这个问题，我们将旋转基数树的扇出从 n 减小到 n^δ 。现在每个内部节点最多包含 n^δ 个球，因此保证至少有 $n^{1-\delta}$ 个内部节点。这确保了始终有至少 $n^{1-\delta} \log n$ 个随机比特影响树的结构。

我们注意到，由于旋转基数树的扇出现在是 n^δ ，每个球由一个对 (s, c) 确定，其中 $s \in [m]$ 是源节点， $c \in [n^\delta]$ 是孩子索引。尽管如此，从球到桶的映射 ϕ 与之前完全一样工作：我们将球 (s, c) 映射到桶 $\phi(s, c) = ((r_s + c) \bmod n)$ ，其中 $r_s \in [n]$ 是随机选取的。

界定溢出球的数量。当然，不同桶 $j \in [n]$ 中的溢出球数量 q_j 之间仍然存在依赖关系。为了处理这些依赖关系，我们使用概率组合学中的一个工具。

称函数 $f: [0, 1]^m \rightarrow \mathbb{R}$ 是 L -Lipschitz 的，如果对于每一对形式为 $x = (x_1, \dots, x_i, \dots, x_m)$ 和 $x' = (x_1, \dots, x'_i, \dots, x_m)$ 的输入，我们有 $|f(x) - f(x')| \leq L$ 。McDiarmid 不等式 [22] 告诉我们，如果 f 是 L -Lipschitz 的，且 $X_1, X_2, \dots, X_m \in [0, 1)$ 是独立随机变量，那么对于任意 $t \geq 0$,

$$\Pr[|f(X_1, \dots, X_m) - \mathbb{E}[f(X_1, \dots, X_m)]| \geq t] \leq 2e^{-2t^2/(mL^2)}.$$

为了将 McDiarmid 不等式应用于我们的情况，定义 $f(r_1, \dots, r_m) := q$ 为溢出球的数量。观察到 f 是 n^δ -Lipschitz 的，因为每个 r_i 可以决定最多 n^δ 个不同球的结果。

由于 $\mathbb{E}[q] = n/\text{poly}(n)$ ，由 McDiarmid 不等式可得

$$\Pr[f(r_1, \dots, r_m) \geq n^{1-\epsilon}] \leq e^{-\Omega(n^{2-2\epsilon}/(mn^{2\delta}))} = e^{-\Omega(n^{2-2\epsilon}/n^{1+2\delta})} = e^{-\Omega(n^{1-4\delta-2\epsilon})}.$$

对于任意 $0 < \epsilon < 1$ ，我们可以设 $\delta = \epsilon/5$ ，使得

$$\Pr[q \geq n^{1-\epsilon}] \leq e^{-\Omega(n^{1-4\delta})} = O\left(n^{-n^{1-\epsilon}}\right).$$

这建立了 (1)。如果我们按照第 2 节的方式实现删除，我们得到以下定理。

定理 2。 $n^{\epsilon/5}$ 又放大旋转基数树是一个随机化线性空间字典，可以一次存储最多 n 个 $(\log n)$ 比特的键/值，并以概率 $1 - O(1/n^{n^{1-\epsilon}})$ 在常数时间内支持每个操作。

我们注意到，从很强的意义上讲，放大旋转基数树是近乎最优的。特别地，对于任意常数 $\epsilon > 0$ ，如果存在一个失败概率为 $1/n^{n^\epsilon}$ 的随机化线性空间字典，那将意味着存在一个确定性（尽管非显式）的线性空间常数时间字典。

引理 3。 令 $\epsilon > 0$ 为任意正常数，假设机器字长为 $w = \Theta(\log n)$ 比特。假设存在一个随机化线性空间字典，可以一次存储最多 n 个 $(\log n)$ 比特的键/值，且失败概率为 $1/n^{n^\epsilon}$ 。那么也存在一个具有相同保证的确定性（不一定是显式的）字典。

证明。为了区分随机化字典和我们正在构造的确定性字典，我们将前者称为哈希表，后者称为字典。

如第 2 节所述，通过每 $O(n)$ 个操作重建我们的字典，我们可以不失一般性地假设字典的生命周期最多为 $O(n)$ 个操作。我们将使用容量为 $n' = cn$ (c 是某个待定的大正常数) 的哈希表来实现字典。这意味着哈希表的失败概率为

$$1/(n')^{(n')^\epsilon} = 1/(cn)^{(cn)^\epsilon}.$$

每个操作发生在一个 $(\log n)$ 比特的键/值对上，因此一个给定操作最多有 $n^{O(1)}$ 种可能的选择。长度为 $O(n)$ 的操作序列的总数因此最多为 $n^{O(n)}$ 。由于我们的哈希表失败概率为 $1/(cn)^{(cn)^\epsilon}$ ，它在任何给定的 $O(n)$ 个操作序列上的总失败概率最多为 $O(n)/(cn)^{(cn)^\epsilon} \leq 1/n^{cn^{\epsilon/2}}$ 。因此，存在某个操作序列使得我们的哈希表不能保持常数时间的概率最多为

$$\frac{n^{O(n)}}{n^{cn^{\epsilon/2}}},$$

如果取 c 为足够大的常数，这个值最多为 $1/2$ 。因此存在某种随机比特的选择，使得我们的哈希表在每个操作序列上都是常数时间的。通过硬编码这种随机比特的选择，我们得到一个确定性常数时间字典。

注意，由于哈希表在 $O(n)$ 个操作上花费的总时间是 $O(n)$ ，它可以使用的随机比特数最多为 $O(nw) = O(n \log n)$ 比特——因此确定性字典可以在线性空间中硬编码这些随机比特。

尽管通常假设机器字长为 $\Theta(\log n)$ 比特，但在机器字长（以及键/值）为某个大小 $w = \omega(\log n)$ 比特的情况下，可实现的最强概率保证是什么，也是一个有趣的问题。一方面，更大的键大小使得引理 3 不再适用，因此原则上，人们可能能够实现 $1/n^{\omega(n)}$ 的失败概率。另一方面，从上限的角度来看，甚至不知道如何在这个设置中实现亚多项式失败概率 [3, 16, 17, 29]。这里，主要障碍似乎不可避免地 与哈希函数有关：能否构造一个从 $[2^w]$ 到 $[\text{poly}(n)]$ 的哈希函数族，使得对于任意给定的 n 元素集合 $S \subseteq [2^w]$ ，我们有 $\max_{x \in S} |\{y \in S \mid h(x) = h(y)\}| \leq \text{polylog } n$ 以概率 $1/n^{\omega(1)}$ 成立？如果这样的族存在，那么它可以直接与定理 2 结合，为任意键大小 w 构造一个实现亚多项式失败概率的字典。我们猜想不存在这样的哈希函数族，并且进而猜想，对于字长 $w = \omega(\log n)$ 比特，亚多项式失败概率是不可能的。

5 5 预算旋转基数树

在本节中，我们提出一个仅使用 $O(\log n \log \log n)$ 随机比特的字典，同时保证每个操作以概率 $1 - 1/\text{poly}(n)$ （即以关于 n 的高概率）在常数时间内完成。我们将这个数据

结构称为**预算旋转基数树** (budget rotated trie)。在附录 A 中, 我们进一步扩展预算旋转基数树, 以支持 $\Theta(\log n)$ 比特的键, 同时仍仅使用 $O(\log n \log \log n)$ 比特的随机性。

我们注意到, 预算旋转基数树实现的保证并不是新颖的——事实上, Dietzfelbinger、Gil、Matias 和 Pippenger [10] 先前的一种方法可以用来在我们正在考虑的 $(\log n)$ 比特键的设置中实现 $O(\log n)$ 随机比特。尽管如此, 我们相信预算旋转基数树的构造本身就很有趣, 既因为它与放大旋转基数树的关系, 也因为它能够以令人惊讶的方式利用逐渐增加独立性的哈希函数。此外, 预算旋转基数树的特定结构将在第 6 节我们追求简洁性时证明是有用的。

我们的出发点再次是旋转基数树, 并且如第 4 节中一样, 我们将树的扇出取为 n^δ (δ 为某个常数); 事实上, 简单地使用 $\delta = 1/4$ 就足够了。

将随机比特数减少到 $O(n/\text{polylog } n)$ 。为了将 n^δ 又旋转基数树转换为预算旋转基数树, 我们的第一个修改是将随机比特数从 $O(n \log n)$ 减少到 $O(n/\text{polylog } n)$ 。当然, 这看起来似乎并不算显著的进展, 但我们稍后将看到这个区别是重要的。

回忆在旋转基数树中, 每个球 b (即内部节点中的每个非空条目) 包含一个指向叶子 (即实际键/值对) 或另一个内部节点 (即孩子) 的指针。我们现在添加第三个选项: 如果球应该指向另一个内部节点 x , 但以 x 为根的子树包含少于 $\Delta = \text{polylog } n$ 个总键, 那么我们将该子树存储为一个动态融合节点 z 。如果以 x 为根的子树的大小随后超过 Δ , 那么我们为 x 创建一个实际的内部节点——在这种情况下, 存储在融合节点 z 中的任何元素保留在 z 中, 而球 b 现在存储两个指针, 一个指向 x , 一个指向 z 。换句话说, 一个球现在有三种可能的状态: 它可以包含一个指向叶子的指针; 它可以包含一个指向动态融合节点的指针; 或者它可以包含两个指针, 一个指向动态融合节点, 一个指向树的另一个内部节点。

这个修改的意义在于, 我们只在以 x 为根的子树包含至少 $\Delta = \text{polylog } n$ 个元素时才创建一个内部节点 x 。重要的是, 这意味着内部节点的总数 m 最多为 $O(n/\Delta) = n/\text{polylog } n$ 。因此, 旋转量 $r_1, r_2, \dots, r_m$ 所需的随机比特数也是 $n/\text{polylog } n$ 。

改变球到桶的映射。我们的下一个修改是改变我们如何将球映射到桶。回忆每个球 b 由一个对 (s, c) 指定, 其中 $s \in [m]$ 是球的源节点, $c \in [n^\delta]$ 是孩子索引。在标准旋转基数树中, 我们使用函数

$$\phi(s, c) = (c + r_s) \bmod n$$

将球映射到桶。

现在我们将改用函数

$$\phi(s, c) = ((c + a_s) \bmod n^\delta) \cdot n^{1-\delta} + b_s,$$

将球映射到桶, 其中 a_s 从 $[n^\delta]$ 中随机选取, b_s 从 $[n^{1-\delta}]$ 中随机选取。

我们可以这样理解 ϕ 。我们将桶分成组 $G_1, \dots, G_{n^\delta}$, 每组大小为 $n^{1-\delta}$, 并使用随机值 $a_s \in [n^\delta]$ 将球分配到一个随机组。一旦球被分配到组 G_i , 它随后被分配到该组中的第 b_s 个桶。重要的是, 分配的设计使得每个源节点 s 最多向任何给定组 G_i 分配它的一个球。在组 G_i 中永远不会有球 b_1, b_2 都从同一个源节点获得它们的 b_s 分配。

由于内部节点数 m 可能高达 $n/\text{polylog } n$, 我们无法负担真正随机地生成 $a_1, a_2, \dots, a_m \in [n^\delta]$ 和 $b_1, b_2, \dots, b_m \in [n^{1-\delta}]$ 。幸运的是, 正如我们现在将看到的, a_i 和 b_i 的角色已被仔细设计, 使得两个序列都可以使用少量的“种子”随机比特生成。

使用 $O(1)$ 独立哈希函数生成 a_i 。 令 k 为一个足够大的正常数, 并从 k 独立哈希函数族中随机选取一个哈希函数 $g: [n] \rightarrow [n^\delta]$ 。由于 $k = O(1)$, 函数 g 可以使用 $O(\log n)$ 随机比特来指定, 并且可以在 $O(1)$ 时间内评估。

我们通过 $a_i := g(i)$ 来计算 a_i 。

为了分析每个组 G_i 中的球数, 我们使用一个众所周知的 k 独立随机变量的尾部界 (参见例如 [2] 或 [12])。

引理 4 ([2] 的引理 2.2)。 令 $k \geq 4$ 为偶数。假设 $X_1, \dots, X_m$ 是 k 向独立的 0-1 随机变量。令 $X = \sum_i X_i$ 。那么, 对于任意 $t \geq 0$,

$$\Pr[|X - \mathbb{E}[X]| \geq t] \leq 2 \left(\frac{mk}{t^2} \right)^{k/2}.$$

定义 X_j 为源节点 j 向组 G_i 发送一个球的事件。 X_j 是 k 独立的, 因此由引理 4 我们有

$$\Pr[|G_i| - \mathbb{E}[|G_i|] \geq n^{0.75}] \leq 2 \left(\frac{kn}{n^{1.5}} \right)^{k/2} \leq n^{-\Omega(k)} = 1/\text{poly}(n).$$

由于 $\delta = 0.25$, 可得

$$\Pr[|G_i| - \mathbb{E}[|G_i|] \geq n^{1-\delta}] \leq 1/\text{poly}(n).$$

由于 $O(n)$ 个球中的每一个等可能地落入任何组, 我们有 $\mathbb{E}[|G_i|] = O(n^{1-\delta})$ 。因此

$$\Pr[|G_i| \leq O(n^{1-\delta})] \geq 1 - 1/\text{poly}(n).$$

也就是说, 每个组 G_i 包含最多 $O(n^{1-\delta})$ 个球, 以关于 n 的高概率成立。

使用增加独立性的哈希函数生成 b_i 。 为了生成 b_i 而不使用大量随机比特, 我们使用一个更复杂的哈希函数族。称哈希函数族 $\mathcal{H}(t)$ (其中 $h: [\text{poly}(t)] \rightarrow [t]$) 是**负载均衡的** (load-balancing), 如果它可以用来将 t 个球映射到 t 个桶, 最大负载为 $\text{polylog } t$; 也就是说, 对于任意固定的大小为 t 的集合 $S \subseteq \text{poly}(t)$, 以及任意固定的 $i \in [t]$, 如果我们选取随机的 $h \in \mathcal{H}$, 那么

$$|\{s \in S \mid h(s) = i\}| \leq \text{polylog } t$$

以概率 $1 - 1/\text{poly}(t)$ 成立。

Celis、Reingold、Segev 和 Wieder [7] 展示了如何构造一个负载均衡的哈希函数族 $\mathcal{H}(t)$ ，使得每个 $h \in \mathcal{H}$ 可以用 $O(\log t \log \log t)$ 随机比特描述，并可以在 $O(\log t \log \log t)$ 时间内评估。族 $\mathcal{H}$ 被称为具有“**逐渐增加的独立性**”，因为每个 $h \in \mathcal{H}$ 实际上是 $O(\log \log t)$ 个具有不同独立性水平的哈希函数 $h_1, \dots, h_{O(\log \log t)}$ 的组合：每个 h_i 确定 h 的 $((3/4)^i \log t)$ 个比特，且每个 h_i 与 $((4/3)^i)$ 独立的距离在 $1/\text{poly } t$ 之内。

族 $\mathcal{H}$ 附带一个权衡。它能够使用 $O(\log t \log \log t)$ 比特实现最大负载 $\text{polylog } t$ （实际上，它甚至实现最大负载 $O(\log t / \log \log t)$ ），但它需要超常数时间来评估。后续工作 [23] 将评估时间从 $O(\log t \log \log t)$ 改进到 $O((\log \log t)^2)$ 。然而，评估时间似乎不太可能改进到 $O(1)$ ，因为仅读取用于评估哈希函数的随机比特就需要 $\Omega(\log \log t)$ 的时间。

超常数的评估时间使得具有逐渐增加独立性的哈希函数不适合直接用于常数时间哈希表 [32]。我们通过将 h 不作为哈希函数，而是作为伪随机数生成器来绕过这个问题。具体来说，我们从 $\mathcal{H}(n^{1-\delta})$ 中随机选取一个 $h : [m] \rightarrow [n^{1-\delta}]$ ，并使用 h 如下初始化 b_i ：

$$b_i := h(i).$$

由于 h 需要 $O((\log \log n)^2)$ 时间来评估，现在每个 b_i 需要 $O((\log \log n)^2)$ 时间来初始化。然而回忆一下，我们只在有超过 $\Delta = \text{polylog } n$ 条记录想要驻留在该节点的子树中时才在我们的旋转基数树中创建一个新的内部节点 x ；前 $\Delta = \text{polylog } n$ 个希望使用 x 的插入被放入一个作为 x 代理的动态融合节点中。因此，我们可以负担花费多达 Δ 时间来初始化节点 x ，并且我们可以将这时间分摊到触发 x 初始化的 Δ 个插入上。由于 $\Delta = \Omega((\log \log n)^2)$ ，我们可以顺利完成 $b_i = h(i)$ 的初始化。

分析最大负载。回忆，以概率 $1 - 1/\text{poly}(n)$ ，每个组 G_i 包含最多 $O(n^{1-\delta})$ 个球。此外，每个球彼此具有不同的源节点。如果一个球的源节点为 s ，那么它被放置在 G_i 的第 b_s 个桶中。

令 $S_i \subseteq [m]$ 是向 G_i 分配球的源节点集合。那么对于每个 $r \in [n^{1-\delta}]$ ， G_i 的第 r 个桶中的球数 $g_{i,r}$ 由下式给出：

$$g_{i,r} = |\{s \in S_i \mid h(s) = r\}|.$$

由于 $h : \text{poly}(n) \rightarrow [n^{1-\delta}]$ 来自一个负载均衡的哈希函数族，我们保证有

$$g_{i,r} \leq \text{polylog } n^{1-\delta} \leq \text{polylog } n$$

以关于 n 的高概率成立。

组合各部分。每个桶最多包含 $\text{polylog } n$ 个球（以高概率）这一事实意味着，与标准旋转基数树一样，每个桶可以用一个动态融合节点来实现。因此，我们字典上的操作以关于 n 的高概率花费 $O(1)$ 时间。如果我们按照第 2 节的方式实现删除，我们得到以下定理。

定理 5。 预算旋转基数树是一个随机化线性空间字典，可以一次存储最多 n 个 $(\log n)$ 比特的键/值，使用 $O(\log n \log \log n)$ 随机比特，并以概率 $1 - 1/\text{poly}(n)$ 在常数时间内支持每个操作。

我们通过观察到预算旋转基数树实现的保证从很强的意义上讲是最优的来结束本节。特别地，如果存在一个失败概率为 $1/n^c$ 但使用少于 $c \log n$ 随机比特的哈希表，那么也必然存在一个确定性的线性空间常数时间字典。

引理 6。 假设存在一个随机化线性空间字典，可以一次存储最多 n 个 $(\log n)$ 比特的键/值，使用 $c \log n$ 随机比特，但失败概率小于 $1/n^c$ 。那么存在一个具有相同保证的确定性字典。

证明。 为了区分随机化字典和我们正在构造的确定性字典，我们将前者称为哈希表。令 R 表示哈希表使用的 $c \log n$ 随机比特。定义 D 为通过设置 $R = 0$ 获得的确定性字典。反设 D 不是常数时间的。那么存在某个操作序列使得 D 上的最后操作需要超常数时间。这意味着，以至少 $1/n^c$ 的概率，同一个操作在我们的哈希表上也会需要超常数时间。但哈希表的失败概率小于 $1/n^c$ ，矛盾。

6 6 实现简洁性

在本节中，我们将注意力转向空间效率。贯穿本节，我们将使用 $c_1 \log n$ 表示每个键的比特大小，使用 $c_2 \log n$ 表示每个值的比特大小，并假设 $c = c_1 + c_2$ 是大于 1 的正常数。这里，与前面各节不同，我们使用 n 表示任意给定时刻的**当前**大小，并且我们允许 n 随时间动态变化，只要键/值长度在初始化后的所有时刻保持为 $\Theta(\log n)$ 比特——这意味着常数 c_1, c_2, c 也动态变化。⁴

⁴ 注意，键的长度至少为 $\log n$ 比特是平凡必要的，因此键/值大小必然是 $\Theta(\log n)$ 。如果我们假设值渐近地不大于键，那么可以通过将字典视为包含额外的 $2^{\varepsilon w}$ 个虚拟元素来强制实现键/值长度 $O(\log n)$ 的界，其中 w 是键/值长度， ε 是一个小正常数。

我们在前面各节中描述的字典在常数因子意义上已经是最优的，使用总共 $\Theta(n \log n)$ 比特来存储 n 个键/值对。现在我们将努力实现最优的空间消耗（在低阶项范围内），即总共使用

$$(1 + o(1)) \log \binom{2^{c \log n}}{n} = cn \log n - n \log n + o(n \log n) \quad (2)$$

比特。这样的字典被称为**简洁的** (succinct) [31]。

事实上，我们将证明一个更一般的结果：任何常数时间字典都可以被转换为一个简洁的常数时间字典，同时（几乎）保持原始字典的随机比特使用和失败概率。

定义一个 $(r(n), p(n), s(n))$ -字典为任何使用 $O(r(n))$ 随机比特、每个操作实现失败概率 $O(p(n))$ 、并使用 $cn \log n - n \log n + O(s(n))$ 比特空间的常数时间字典。由于我们对在键数变化时自动调整大小的字典感兴趣，应该将 $r(n)$ 、 $p(n)$ 和 $s(n)$ 视为函数而非

固定值。注意，在空间高效且时间高效字典的语境中，失败事件可以是关于时间的（一个操作需要 $\Omega(1)$ 时间）或关于空间的（字典未能适应 $cn \log n - n \log n + O(s(n))$ 比特）——失败概率 $p(n)$ 意味着在任意给定时刻，发生失败的概率应该最多为 $O(p(n))$ 。

本节的主要结果可以陈述如下。

定理 7. 令 ϵ 为一个小的正常数。假设 $r(n)$ 和 $p(n)$ 是非递减函数，满足 $r(n) \leq O(n)$ 和 $\exp(-n^{1-\epsilon}) \leq p(n) \leq 1/\text{polylog}(n)$ 。给定一个 $(r(n), p(n), n \log n)$ -字典，可以构造一个 $(r'(n), p'(n), s'(n))$ -字典，其中

$$r'(n) = r(n) + (\log p(n)^{-1}) \cdot (\log \log n)^3,$$

$$p'(n) = p(n / \log \log n),$$

且

$$s'(n) = \frac{n \log n}{\log \log n}.$$

将定理 7 应用于定理 2 和 5，我们得到以下定理的简洁版本。

推论 8. 令 $0 < \epsilon < 1$ 为一个正常数。存在一个 $(r(n), p(n), s(n))$ -字典，使用 $r(n) = O(n)$ 随机比特，产生失败概率 $p(n) = \exp(-n^{1-\epsilon})$ ，并产生与信息论最优值相比 $s(n) = O\left(\frac{n \log n}{\log \log n}\right)$ 比特的附加空间开销。

推论 9. 存在一个 $(r(n), p(n), s(n))$ -字典，使用 $r(n) = O(\log n (\log \log n)^3) = \tilde{O}(\log n)$ 随机比特，产生失败概率 $p(n) = 1/\text{poly}(n)$ ，并产生与信息论最优值相比 $s(n) = O\left(\frac{n \log n}{\log \log n}\right)$ 比特的附加空间开销。

本节的其余部分将用于证明定理 7。我们在第 6.1 节开始，介绍 Raman 和 Rao [31] 的一个归约，它将构造简洁字典的问题转换为另一个问题，我们称之为**多集合问题** (many-sets problem)。然后，在第 6.2 节中，我们展示如何使用一个 $(r(n), p(n), O(n \log n))$ -字典来解决多集合问题，同时近似保持字典的随机性保证——我们解决方案的一个有趣特点是它广泛使用了第 5 节中构造的预算旋转基数树。

6.1 6.1 到多集合问题的归约

我们证明定理 7 的一个重要工具是 Raman 和 Rao [31] 的一个归约。该归约将构造简洁动态字典的问题转化为一个不同的问题，我们称之为**多集合问题**。

令 $\epsilon > 0$ 为我们可以选择的一个小正常数。参数为 ϵ 的多集合问题定义如下。令 $S_1, S_2, \dots, S_m$ 为（动态变化的） $O(\log n)$ 比特键/值对的集合，满足 $\sum_i |S_i| = n$ ， $m \leq n/\text{polylog}(n)$ ，且对所有 $i \in [m]$ 有 $|S_i| \leq n^\epsilon$ ，其中 n 和 m 允许随时间动态演化。我们将使用 $\ell_i = O(\log n)$ 表示 S_i 中每个键/值对的大小（因此不同的 S_i 可能有不同大小的键/值对）。

多集合问题的任何解必须在常数时间内支持以下操作：

- INSERT(i, x, y), 将键/值对 (x, y) 插入 S_i ;
- DELETE(i, x), 从 S_i 中删除 x (及其对应的值);
- 以及 QUERY(i, x), 返回 S_i 中与 x 关联的值, 或声明 $x \notin S_i$ 。

一个多集合问题的解被称为 $(r(n), p(n), s(n))$ -解, 如果它使用 $r(n)$ 随机比特, 每次插入的失败概率为 $p(n)$, 并使用总空间

$$\sum_i |S_i| \cdot (\ell_i + O(\log |S_i|)) + s(n)$$

比特。

以下归约 (隐式地) 在 [31] 的第 3 节中给出：

定理 10. 构造一个 $(r(n), p(n), s(n) + n \log n / \log \log n)$ -字典的问题确定性地归约到构造多集合问题的一个 $(r(n), p(n), s(n))$ -解的问题, 其中参数 ε 是我们选择的一个正常数。

理解该归约。 虽然我们将定理 10 的完整证明推迟到 [31], 但我们在此花一点时间简要描述其高层次结构。关键的洞察是以一种聪明的方式使用基数树 (这与本文其他地方使用基数树的方式有实质性的不同)。树的不同节点有不同的扇出: 如果一个节点包含 $r > \text{polylog } n$ 个键且在树中的深度为 $O(1)$, 则其扇出为 $r / \text{polylog } n$ 。如果一个节点包含 $r \leq \text{polylog } n$ 个键, 或者在树中具有足够大的常数深度, 则该节点是叶子, 且该节点的元素对应于多集合问题中的一个集合 S_i 。

与本文中的数据结构不同, 这棵树的内部节点使用直接的数组来实现: 如果一个节点的扇出为 f , 则使用一个大小为 f 的数组来实现它。幸运的是, 通过选择扇出 f 为 $r / \text{polylog } n$ (其中 r 是该节点存储的元素数量), 可以确保用于实现内部节点的数组的累计大小为 $n / \text{polylog } n$ 。

一个关键的洞察是, 这棵树允许我们从键中”削去”比特: 如果一个节点 x 的扇出为 f , 那么我们可以从存储在 x 的孩子中的每个键中移除高位的 $\log f$ 个比特 (这些比特现在隐式地存储在树的路径中)。Raman 和 Rao [31] 证明, 如果一个叶子的大小为 $|S_i|$, 那么 S_i 中每个键/值对的长度 ℓ_i 将满足 $\ell_i \leq c \log n - \log n - q \log |S_i| + O(\log \log n)$ 比特, 其中 q 是我们选择的一个正常数 (取决于树的最大深度)。这就是为什么对于多集合问题, 对 S_i 中的每个键使用 (摊销) 空间 $\ell_i + O(\log |S_i|)$ 比特是可以接受的。

我们注意到, 在 [31] 中, 定理 10 被证明具有摊销的常数时间操作。摊销来自于这样一个事实: 每当树中一个节点 x 的大小发生实质性变化时, 它必须被重建; 然而, 通过将重建分摊到 $\Theta(|S_i|)$ 个操作 (每个操作都修改节点 x) 上, 重建可以轻松地去摊销化为每个操作 $O(1)$ 时间 (且不损害空间效率, 因为每个元素在任何时刻只存在于节点的一个版本中)。

在继续之前，值得花一点时间理解多集合问题中假设的界 $m \leq n/\text{polylog}(n)$ 和 $|S_i| \leq n^\varepsilon$ 从何而来。界 $m \leq n/\text{polylog}(n)$ 来自于树的每个内部节点的扇出为 $r/\text{polylog}(n)$ (r 是其大小)；由于树的深度为 $O(1)$ ，这意味着叶子的总数（从而 S_i 的数量）最多为 $O(n/\text{polylog } n)$ 。界 $|S_i| \leq n^\varepsilon$ 来自于以下事实：如果某个 S_i 的大小 $> n^\varepsilon$ ，那么在其根到叶路径上的每个节点处，集合中的每个元素将被削去 $\geq \log n - O(\log \log n)$ 个比特；但假设树具有足够大的常数深度，这意味着 S_i 的元素的所有比特都被削去了，这是一个矛盾。

一个出发点：Raman 和 Rao 对多集合问题的解。 Raman 和 Rao [31] 给出了多集合问题的一个简单解，该解每次操作产生常数期望时间，并且也将作为我们构造的出发点。在高层次上，每个 S_i 被存储在一个由两部分组成的结构中，包括一个**骨架** (skeleton) A_i 和一个**存储数组** (storage array) B_i 。⁵

存储数组 B_i 在逻辑上被实现为一个动态调整大小的数组，该数组连续地存储 S_i 的元素（包括键和值）。 $\{B_i\}$ 的动态调整大小可以被实现（并去摊销化）为每个操作 $O(1)$ 时间，同时确保总体上 B_i 数组使用的空间在最优值的 $1 + 1/\text{polylog}(n)$ 因子内 [31]（关于简洁动态数组的更详细处理，另见 [6]）。

骨架 A_i 允许对 S_i 的查询在 B_i 中找到相应的键/值。特别地，骨架是一个字典，将 S_i 中每个键 x 的 $(\log |S_i|)$ 比特哈希 $h(x)$ 映射到 x 在 B_i 中出现的位置索引 j 。⁶

⁵ “骨架”和“存储数组”这两个名称是我们为了方便讨论而在此建立的约定，在原文 [31] 中并未使用。

⁶ 如 [31] 所指出，有一个关于删除的微妙问题必须小心处理。每当发生删除时， B_i 的某个位置 j 会产生一个空隙。为了消除这个空隙，必须将 B_i 中的最后一个元素 x 移动到位置 j 。这意味着我们还必须将元素 x 在 A_i 中存储的索引更新为 j 。

有利的一面是，在骨架 A_i 内部，键（即哈希 $h(x)$ ）和值（即 B_i 中的索引）都只有 $O(\log |S_i|)$ 比特——这意味着 A_i 可以使用任何标准的线性空间字典来实现，而不必担心简洁性。不利的一面是，向 A_i 的插入可能以两种方式失败：（1）被插入元素的哈希 $h(x)$ 满足 $h(x) = h(x')$ （对某个其他 $x' \in S_i$ ）；或（2）用于实现 A_i 的字典失败。

假设 A_i 被实现为具有 $1/\text{poly}(|S_i|)$ 的失败概率（即为一个高概率字典），那么任何给定向 S_i 的插入失败的概率是 $1/\text{poly}(|S_i|)$ 。当然，每个 $|S_i|$ 可以任意小，这就是为什么 [31] 中给出的数据结构提供常数期望时间操作，而不是高概率保证。

6.2 6.2 定理 7 的证明

通过前一子节中的归约（定理 10），我们只需证明以下命题：

命题 11. 令 ε 为一个小的正常数。假设 $r(n)$ 和 $p(n)$ 是非递减函数，满足 $r(n) \leq O(n)$ 和 $\exp(-n^{1-\varepsilon}) \leq p(n) \leq 1/\text{polylog}(n)$ 。给定一个 $(r(n), p(n), n \log n)$ -字典，可以构造多集合问题的一个 $(r'(n), p'(n), s'(n))$ -解，参数为 $\tilde{\varepsilon} = \varepsilon/2$ ，其中

$$r'(n) = r(n) + (\log p(n)^{-1}) \cdot (\log \log n)^3,$$

$$p'(n) = p(n / \log \log n),$$

且

$$s'(n) = \frac{n \log n}{\log \log n}.$$

我们将通过改编前一节中概述的骨架/存储数组方法来证明命题 11。基本思路是将每个 A_i 实现为一个预算旋转基数树，然后以恰到好处的方式在 A_i 之间共享随机比特，使得 (1) 使用的随机比特总数很小；但 (2) 以良好的概率，在任何给定时刻存在的元素中只有 $O(1 / \log \log n)$ 比例的元素在插入时经历了失败。然后我们将经历失败的元素存储在由给定的 $(r(n), p(n), O(n \log n))$ -字典实现的后花园 (backyard) 数据结构中。

讨论 A_i 的预备知识。由于给定骨架 A_i 的大小可能随时间波动，骨架 A_i 在每次其大小变化常数因子时需要被重建。这些重建可以去摊销化为每个操作常数时间（并且由于字典 A_i 被允许使用 $O(|S_i| \log |S_i|)$ 比特，重建给 A_i 增加常数因子的空间开销是可以接受的）。

在任何给定时刻，定义 S_i 的**骨架大小** (skeletal size) a_i 为 A_i 最近一次被（逻辑上）重建时 S_i 的大小。骨架大小始终满足 $a_i = \Theta(|S_i|)$ ，但具有仅在重建发生时才变化的方便性质。骨架大小 a_i 也将用于决定我们如何实现 A_i （具有不同骨架大小的集合可能彼此以不同方式实现）——这意味着，每次 A_i 被重建（且 a_i 变化）时， A_i 的实现也可能变化。

定义后花园结构。数据结构的一个重要组成部分将是一个**后花园字典** (backyard dictionary) T ，它用于以空间低效的方式存储少量键/值。我们使用给定的 $(r(n), p(n), n \log n)$ -字典来实现 T ，并且我们不失一般性地假设 $|T| \geq n / \log \log n$ 始终成立（如果 $|T| < n / \log \log n$ ，我们可以用虚拟元素填充以增大其大小）。这意味着，在任何给定时刻， T 最多使用 $O(r(n))$ 随机比特，失败概率最多为 $O(p(n / \log \log n))$ ，并使用最多

$$O((|T| + n / \log \log n) \cdot \log n)$$

比特的空间。为了使 T 满足命题 11 的约束，我们需要证明，以概率 $1 - O(p'(n))$ ，在任何给定时刻有

$$|T| \leq O(n / \log \log n). \quad (3)$$

值得花一点时间精确描述 T 为属于给定骨架 A_i 的每个键/值对 (x, y) 存储什么信息：它将 (i, x) 作为键存储，将 y 作为值存储。由于不同的 A_i 有不同的键/值大小（都是 $\Theta(\log n)$ ），我们在后花园中将键/值的大小填充为统一的固定长度 $\Theta(\log n)$ 。⁷

⁷ 还有一个额外的微妙之处：每当一个元素被存储在后花园中时，它也应该被存储在相应的存储数组 B_i 中，这样每当 A_i 被重建时，它就可以确定其哪些元素当前在后花园中，从后花园中移除那些元素，并将这些元素包含在 A_i 的重建中。

自动将小的 A_i 存储在后花园中。 令 c 为一个足够大的正常数。我们通过简单地将满足 $a_i \leq \log^c n$ 的 A_i 的所有元素放入后花园 T 来处理它们。这样的 A_i 的数量显然最多为 m ，根据多集合问题的定义， m 最多为 $n/\text{polylog } n \leq O(n/\log^{2c} n)$ 。因此这些 A_i 贡献给后花园的元素数量最多为 $O(\log^c n \cdot n/\log^{2c} n) = o(n/\log^c n)$ 。在本节的其余部分，我们将专注于满足 $a_i > \log^c n$ 的 A_i 。

将剩余的 A_i 划分为组 $G_{j,k}$ 。 我们称每个 A_i 属于类别 $j = \lfloor \log a_i \rfloor$ 。在每个类别 j 内，我们将满足 $a_i > \log^c n$ 的 A_i 划分为

$$t_j = \Theta\left(\frac{(\log \log n)^2 \log p(n)^{-1}}{j}\right) \quad (4)$$

个组 $G_{j,1}, G_{j,2}, \dots, G_{j,t_j}$ ，使得对于每个组 $k \in [t_j]$ ，

$$\sum_{A_i \in G_{j,k}} a_i \leq O(n/t_j).$$

注意这样的划分是可行的当且仅当我们可以保证对每个 i 有 $a_i \leq O(n/t_j)$ ——幸运的是，这由以下推导得出：

$$\begin{aligned} a_i \leq n^{\tilde{\varepsilon}} &\leq O\left(\frac{n}{n^{1-\tilde{\varepsilon}}}\right) \leq O\left(\frac{n}{(\log \log n)^2}\right) \quad (\text{因为 } \tilde{\varepsilon} = \varepsilon/2, \text{ 且 } \exp(-n^{1-\varepsilon}) \leq p(n)) \\ &\leq O\left(\frac{n \cdot (\log \log n)^2 \log p(n)^{-1}}{j}\right) \\ &= O(n/t_j). \end{aligned}$$

对于任何给定的类别 j ，我们实现每个组 $G_{j,1}, G_{j,2}, \dots, G_{j,t_j}$ 的唯一区别是每个组 $G_{j,k}$ 使用不同的随机比特序列 $R_{j,k}$ （长度为 $\Theta(j \log j)$ ）。

实现每个 A_i 。 现在我们描述如何使用 $\Theta(j \log j) = \Theta(\log a_j \log \log a_j) = \Theta(\log |A_j| \log \log |A_j|)$ 个随机比特 $R_{j,k}$ 来实现给定的骨架 $A_i \in G_{j,k}$ 。回忆我们将满足 $a_i \leq \log^c n$ 的 A_i 存储在后花园中，因此这里我们只需关注满足 $a_i > \log^c n$ 的 A_i 。

定义一个哈希函数 $h_{j,k}$ ，将元素 $x \in A_i$ 映射到 $(\log |A_i|)$ 比特的字符串 $h(x)$ 。该哈希函数使用附录 A 引理 13 中给出的哈希函数族来实现，这意味着该哈希函数使用 $\Theta(\log |A_j|)$ 随机比特，并且在 A_i 上避免冲突的概率为 $1 - 1/\text{poly}(|A_i|)$ （注意，由于 $a_i > \log^c n$ ，引理的前提条件得以满足）。骨架 A_i 将哈希 $h_{j,k}(x)$ （对 $x \in A_i$ ）映射到 B_i 中的索引。

我们使用预算旋转基数树 (定理 5) 来实现字典 A_i ——这使用 $\Theta(\log |A_j| \log \log |A_j|)$ 随机比特, 且每次插入的失败概率为 $1/\text{poly}(|A_j|)$ 。我们注意到, 除了利用预算旋转基数树提供的概率保证之外, 我们还将利用该数据结构经历失败的特别简单的方式: 唯一可能的失败模式是树中的某个桶溢出。这将允许我们优雅地处理 A_j 失败的情况: 我们将经历失败的元素存储在一个备份数据结构中, 并允许导致失败的 A_j 继续运行, 就像那次插入从未发生过一样。

更详细地说, 向 A_i 的插入可能以两种原因失败: (1) 被插入元素的哈希 $h_{j,k}(x)$ 满足 $h_{j,k}(x) = h_{j,k}(x')$ (对某个其他 $x' \in S_i$); 或 (2) 用于实现 A_i 的预算旋转基数树失败 (即该插入会导致树中的某个桶溢出)。这些事件中任何一个在任何给定插入时发生的概率是 $1/\text{poly}(|A_i|) = 1/\text{poly}(2^j)$ 。每当向 A_i 的插入失败时, 我们将键/值对改为存储在后花园数据结构 T 中。这意味着我们的完整数据结构有两种可能的失败模式: T 自身失败的情况, 以及 T 变得过大而违反 (3) 的情况。

分析失败概率。后花园数据结构 T 按设计具有最多 $O(p(n/\log \log n))$ 的失败概率。因此, 我们的任务是界定 (3) 失败的概率。

引理 12。以概率 $1 - \text{poly}(p(n))$, 类别 j 在任何给定时刻向后花园 T 贡献最多 $O(n/(\log \log n)^2)$ 个元素。

证明。我们已经确立了满足 $a_i \leq \log^c n$ 的 A_i 确定性地向后花园贡献最多 $O(n/(\log \log n)^2)$ 个元素 (因为它们确定性地最多有 $O(n/(\log \log n)^2)$ 个元素)。因此我们这里专注于满足 $a_i > \log^c n$ 的 A_i (注意这些都属于满足 $j > \Omega(\log \log n)$ 的类别)。

现在考虑一个满足 $j = \Omega(\log \log n)$ 的类别 j 。作为简写, 我们将使用 t 表示 t_j , G_k 表示 $G_{j,k}$ 。对于每个组 G_k ($k \in [t]$), 定义 X_k 为 G_k 中的骨架贡献给后花园的元素数量。我们有 $X_k \leq O(n/t)$ 确定性地成立; 并且, 由于 $A_i \in G_k$ 中的每个元素 x 具有 $1/\text{poly}(2^j)$ 的失败概率, 我们有

$$\mathbb{E}[X_k] = \frac{n/t}{\text{poly}(2^j)}. \quad (5)$$

最后, 由于每个组 $G_1, G_2, \dots, G_t$ 使用不同的随机比特序列, X_k 是相互独立的。定义 $X'_k = X_k/(n/t)$, 则 G_k 贡献给后花园的元素数量可以表示为 $\frac{n}{t} \cdot Y_j$, 其中

$$Y_j = \sum_{k=1}^t X'_k.$$

X'_k 是 $[0, 1]$ 范围内的独立随机变量, 且由 (5) 有

$$\mathbb{E}[Y_j] = \sum_{k=1}^t \mathbb{E}[X'_k] = \sum_{k=1}^t (t/n) \mathbb{E}[X_k] = t/\text{poly}(2^j). \quad (6)$$

对 Y_j 应用 Chernoff 界, 我们得到

$$\Pr[Y_j > (1 + D)\mathbb{E}[Y_j]] \leq \left(\frac{e^D}{(1 + D)^{1+D}} \right)^{\mathbb{E}[Y_j]},$$

对于 $D \geq \Omega(1)$, 这意味着

$$\Pr[Y_j > D \cdot \mathbb{E}[Y_j]] \leq D^{-\Omega(D \cdot \mathbb{E}[Y_j])}.$$

设

$$D = \frac{t}{(\log \log n)^2 \mathbb{E}[Y_j]}$$

$$= \frac{\text{poly}(2^j)}{(\log \log n)^2} \quad (\text{由 (6)})$$

$$\geq 2^{\Omega(j)} \quad (\text{因为 } j \geq \Omega(\log \log n)),$$

其中最后一个关系使用了 $j \geq \Omega(\log \log n)$ 。那么我们有

$$\Pr \left[Y_j > \frac{t}{(\log \log n)^2} \right] \leq D^{-\Omega(D \cdot \mathbb{E}[Y_j])}$$

$$= \exp(-\Omega(j \cdot (D \cdot \mathbb{E}[Y_j])))$$

$$= \exp(-\Omega(j \cdot (t/(\log \log n)^2))) \quad (\text{因为 } D \geq 2^{\Omega(j)})$$

$$= \exp(-\Omega(j \cdot (\log p(n)^{-1}/j))) \quad (\text{由 (4)})$$

$$= \exp(-\Omega(\log p(n)^{-1}))$$

$$= \text{poly}(p(n)).$$

因此, 我们以概率 $1 - \text{poly}(p(n))$ 有

$$Y_j \leq \frac{t}{(\log \log n)^2}.$$

类别 j 贡献给后花园的元素数量因此最多为

$$O\left(\frac{n}{t} \cdot Y_j\right) \leq O(n/(\log \log n)^2),$$

如所需。

综合各部分, T 的总大小为 $O(n/\log \log n)$, 其概率为

$$1 - O(p(n/\log \log n)) - O(\log \log n) \cdot \text{poly}(p(n)) = 1 - O(p(n/\log \log n)).$$

界定随机比特数。最后, 我们计算数据结构使用的随机比特数。后花园使用 $O(r(n))$ 随机比特, 因此只需要界定每个类别中骨架使用的随机比特数。注意不同类别可以使用彼此相同的随机比特 (因为我们不要求类别之间的独立性), 因此只需界定任何给定骨架类别使用的随机比特数。在类别 j 中, 有 t_j 个组, 每个组使用 $\Theta(j \log j) = O(j \log \log n)$ 随机比特。因此该类别使用的随机比特总数为

$$O(j(\log \log n)t_j) = O\left(j \log \log n \cdot \frac{(\log \log n)^2 \log p(n)^{-1}}{j}\right) = O((\log \log n)^3 \log p(n)^{-1}).$$

总的来说, 数据结构使用的随机比特数为

$$O(r(n)) + O((\log \log n)^3 \log p(n)^{-1}).$$

这完成了命题 11 的证明, 从而也完成了定理 7 的证明。

7 附录 A: 使用 $O(\log n)$ 随机比特的全空间归约

在本节中, 我们将预算旋转基数树扩展到支持来自超多项式大小全空间 U 的键。贯穿本节, 我们设 $U = [2^u]$ ($u = n^{o(1)}$), 并假设机器字长为 $\Omega(u)$ 比特。

为了支持大键, 自然的方法是首先将元素从 U 哈希到一个多项式大小的较小全空间 U' , 然后将 $(\log n)$ 比特的键与指向完整键/值的指针一起存储在哈希表中。过去关于负载均衡哈希函数的工作 [7] 使用了一个两两独立的哈希函数 $h: [2^u] \rightarrow [\text{poly}(n)]$ 来执行这个归约。这需要使用 $\Omega(u)$ 随机比特, 当 u 很大时, 这显著大于 $\log n \log \log n$ 。

相比两两独立哈希函数, 改用 Pagh 的构造 [28] (该构造又基于 Fredman、Komlós 和 Szemerédi [14] 的早期构造) 是一个有吸引力的替代方案, 该构造给出仅需要 $O(\log n + \log \log u)$ 随机比特的 $(1 + o(1))$ -全域哈希函数。这个构造唯一的不足之处在于它不是完全显式的。该构造需要访问一个随机素数 $p \in [\text{poly}(n)]$, 但构造这样一个素数的唯一已知的时间高效高概率方法需要 $\Omega(\log n \log \log n)$ 随机比特 (参见 [13] 中的讨论)。

幸运的是, 这个问题相对容易解决。为了完整性, 我们现在给出一个简单的哈希函数族的构造, 该族可以在 $o(n)$ 时间内初始化并用于全空间归约。

引理 13。令 $n > u^c$ (c 为足够大的正常数), 令 $S \subseteq [2^u]$ 为大小为 n 的集合。令 P 为范围 $[n^{2/c}]$ 中的素数集合。从 P 中独立且均匀随机地选取 $p_1, p_2, \dots, p_{c^2}$, 并定义函数 $h: [2^u] \rightarrow [n^{2c}]$ 为

$$h(x) = (x \bmod p_1 p_2 \cdots p_{c^2}).$$

以概率 $1 - 1/\text{poly}(n)$, h 在 S 上是单射。

证明。 $|h(S)| = S$ 的概率满足

$$\Pr[|h(S)| \neq S] \leq \sum_{s_1, s_2 \in S} \Pr[|s_1 - s_2| \text{ 能被所有 } p_1, p_2, \dots, p_{c^2} \text{ 整除}],$$

其中 s_1 和 s_2 隐式地取为不同元素。由于 p_i 是独立的, 这等于

$$\sum_{s_1, s_2 \in S} (\Pr[|s_1 - s_2| \text{ 能被 } p_1 \text{ 整除}])^{c^2}.$$

$|s_1 - s_2|$ 是 $U = [2^u]$ 中的元素, 因此最多有 u 个不同的素因子。所以,

$$\Pr[|h(S)| \neq S] \leq \sum_{s_1, s_2 \in S} \left(\frac{u}{|P|} \right)^{c^2} \leq \sum_{s_1, s_2 \in S} \left(\frac{n^{1/c}}{|P|} \right)^{c^2}.$$

由素数定理, 范围 $[n^{2/c}]$ 中的素数集合 P 的大小为 $\Omega(n^{2/c}/\log n)$ 。因此,

$$\begin{aligned} \Pr[|h(S)| \neq S] &\leq O\left(\sum_{s_1, s_2 \in S} \left(\frac{\log n}{n^{1/c}}\right)^{c^2}\right) \\ &\leq O\left(\sum_{s_1, s_2 \in S} \frac{\log^{c^2} n}{n^c}\right) \\ &\leq O\left(\frac{n^2 \log^{c^2} n}{n^c}\right) \\ &\leq 1/\text{poly}(n). \end{aligned}$$

由于 $[n]$ 中的所有素数可以在 $O(n^2)$ 时间内枚举, 我们得到以下推论:

推论 14。 令 $u = n^{o(1)}$ 。对于任意常数 $\epsilon > 0$, 存在一个显式的常数时间哈希函数族 $\mathcal{H}$, 其中 $h: [2^u] \rightarrow [\text{poly}(n)]$, 使得 (a) 随机函数 $h \in \mathcal{H}$ 可以使用 $O(\log n)$ 随机比特在 $O(n^\epsilon)$ 时间内构造; (b) 对于任意固定的大小为 n 的集合 $S \subseteq U$, 对于随机的 $h \in \mathcal{H}$, 我们有 $|h(S)| = |S|$ 以概率 $1 - 1/\text{poly}(n)$ 成立。

我们可以使用推论 14 来构造支持大字集 (large universes) 的预算旋转基数树的一个版本。

定理 15。 令 $u = n^{o(1)}$, 假设键/值为 u 比特, 并假设机器字长至少为 $\Omega(u)$ 比特。预算旋转基数树使用 $O(\log n \log \log n)$ 随机比特, 使用 $O(nu)$ 比特空间, 并支持一次对最多 n 个键/值的插入/删除/查询操作。数据结构可以在 $O(n^\epsilon)$ 时间内初始化 (ϵ 为我们选择的正常数), 且每个插入/删除/查询操作以概率 $1 - 1/\text{poly}(n)$ 花费常数时间。

为了消除 $O(n^\epsilon)$ 的初始化成本，我们还可以构造同一数据结构的动态版本，其中存在某个数据结构大小的上界 N ，但真实大小 n 随时间变化。每当数据结构的大小变化常数因子时，我们基于新的 n 值重建它。每次重建花费 $O(n^\epsilon)$ 时间（以关于 n 的高概率），但重建的成本可以分摊到 $\Omega(n)$ 个操作上。这个新数据结构的性质可以用以下推论总结。

推论 16。 令 $u = N^{o(1)}$ ，假设键/值为 u 比特，并假设机器字长至少为 $\Omega(u)$ 比特。动态预算旋转基数树使用 $O(\log N \log \log N)$ 随机比特，并支持一次对最多 N 个键/值的插入/删除/查询操作。如果它正在存储 n 个键/值对，那么它使用 $O(nu)$ 比特空间，且每个插入/删除/查询操作以概率 $1 - 1/\text{poly}(n)$ 花费常数时间。

8 致谢

作者感谢 Martin Dietzfelbinger 提供的关于先前工作的有益指引，并感谢 Michael A. Bender 和 Martín Farach-Colton 进行的多次富有洞见的技术讨论。

参考文献

- [1] Alfred V Aho and John E Hopcroft. The design and analysis of computer algorithms. Pearson Education India, 1974.
- [2] Mihir Bellare and John Rompel. Randomness-efficient oblivious sampling. In *Proceedings 35th Annual Symposium on Foundations of Computer Science (FOCS)*, pages 276–287. IEEE, 1994.
- [3] Michael A Bender, Alex Conway, Martín Farach-Colton, William Kuszmaul, and Guido Tagliavini. All-purpose hashing. *arXiv preprint arXiv:2109.04548*, 2021.
- [4] Michael A. Bender, Martín Farach-Colton, John Kuszmaul, William Kuszmaul, and Mingmou Liu. On the optimal time/space tradeoff for hash tables. In *Proceedings of the Fifty-Fourth Annual ACM Symposium on Theory of Computing (STOC)*, 2022.
- [5] Jon Bentley. *Programming pearls*. Addison-Wesley Professional, 2016.
- [6] Andrej Brodnik, Svante Carlsson, Erik D Demaine, J Ian Ian Munro, and Robert Sedgwick. Resizable arrays in optimal time and space. In *Workshop on Algorithms and Data Structures*, pages 37–48. Springer, 1999.
- [7] L Elisa Celis, Omer Reingold, Gil Segev, and Udi Wieder. Balls and bins: Smaller hash families and faster evaluation. In *Proceedings of the 2011 IEEE 52nd Annual Symposium on Foundations of Computer Science (FOCS)*, pages 599–608, 2011.
- [8] M Dietzfelbinger, A Karlin, K Mehlhorn, FM auf der Heide, H Rohnert, and RE Tarjan. Dynamic perfect hashing: upper and lower bounds. In *Proceedings of the 29th Annual Symposium on Foundations of Computer Science (FOCS)*, pages 524–531, 1988.

- [9] Martin Dietzfelbinger and Friedhelm Meyer auf der Heide. A new universal class of hash functions and dynamic hashing in real time. In *International Colloquium on Automata, Languages, and Programming* (ICALP), pages 6–19. Springer, 1990.
- [10] Martin Dietzfelbinger, Joseph Gil, Yossi Matias, and Nicholas Pippenger. Polynomial hash functions are reliable. In *International Colloquium on Automata, Languages, and Programming*, pages 235–246. Springer, 1992.
- [11] Martin Dietzfelbinger and Philipp Woelfel. Almost random graphs with simple hash functions. In *Proceedings of the Thirty-Fifth Annual ACM Symposium on Theory of Computing* (STOC), pages 629–638, 2003.
- [12] Devdatt P Dubhashi and Alessandro Panconesi. *Concentration of measure for the analysis of randomized algorithms*. Cambridge University Press, 2009.
- [13] Pierre-Alain Fouque and Mehdi Tibouchi. Close to uniform prime number generation with fewer random bits. In *International Colloquium on Automata, Languages, and Programming* (ICALP), pages 991–1002. Springer, 2014.
- [14] Michael L. Fredman, Janos Komlos, and Endre Szemerédi. Storing a sparse table with $O(1)$ worst case access time. In *Proceedings of the 23rd Annual Symposium on Foundations of Computer Science* (FOCS), pages 165–169. IEEE Computer Society, 1982.
- [15] Michael L Fredman and Dan E Willard. Blasting through the information theoretic barrier with fusion trees. In *Proceedings of the Twenty-Second Annual ACM Symposium on Theory of Computing* (STOC), pages 1–7, 1990.
- [16] Michael T Goodrich, Daniel S Hirschberg, Michael Mitzenmacher, and Justin Thaler. Fully de-amortized cuckoo hashing for cache-oblivious dictionaries and multimaps. *arXiv preprint arXiv:1107.4378*, 2011.
- [17] Michael T Goodrich, Daniel S Hirschberg, Michael Mitzenmacher, and Justin Thaler. Cache-oblivious dictionaries and multimaps with negligible failure probability. In *Mediterranean Conference on Algorithms*, pages 203–218. Springer, 2012.
- [18] Torben Hagerup, Peter Bro Miltersen, and Rasmus Pagh. Deterministic dictionaries. *Journal of Algorithms*, 41(1):69–85, 2001.
- [19] Eyal Kaplan, Moni Naor, and Omer Reingold. Derandomized constructions of k -wise (almost) independent permutations. *Algorithmica*, 55(1):113–133, 2009.
- [20] Mingmou Liu, Yitong Yin, and Huacheng Yu. Succinct filters for sets of unknown sizes. In *47th International Colloquium on Automata, Languages, and Programming* (ICALP). Schloss Dagstuhl-Leibniz-Zentrum für Informatik, 2020.
- [21] Michael Luby and Charles Rackoff. How to construct pseudorandom permutations from pseudorandom functions. *SIAM Journal on Computing*, 17(2):373–386, 1988.

- [22] Colin McDiarmid. On the method of bounded differences. *Surveys in combinatorics*, 141(1):148–188, 1989.
- [23] Raghu Meka, Omer Reingold, Guy N Rothblum, and Ron D Rothblum. Fast pseudorandomness for independence and load balancing. In *International Colloquium on Automata, Languages, and Programming (ICALP)*, pages 859–870. Springer, 2014.
- [24] Moni Naor and Omer Reingold. On the construction of pseudorandom permutations: Luby–Rackoff revisited. *Journal of Cryptology*, 12(1):29–66, 1999.
- [25] Anna Ostlin and Rasmus Pagh. Uniform hashing in constant time and linear space. In *Proceedings of the Thirty-Fifth Annual ACM Symposium on Theory of Computing (STOC)*, pages 622–628, 2003.
- [26] Anna Pagh and Rasmus Pagh. Uniform hashing in constant time and optimal space. *SIAM Journal on Computing*, 38(1):85–96, 2008.
- [27] Rasmus Pagh. Faster deterministic dictionaries. In *Proceedings of the Eleventh Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 487–493, USA, 2000.
- [28] Rasmus Pagh. Dispersing hash functions. *Random Structures & Algorithms*, 35(1):70–82, 2009.
- [29] Mihai Pătraşcu and Mikkel Thorup. The power of simple tabulation hashing. *Journal of the ACM (JACM)*, 59(3):1–50, 2012.
- [30] Mihai Pătraşcu and Mikkel Thorup. Dynamic integer sets with optimal rank, select, and predecessor search. In *2014 IEEE 55th Annual Symposium on Foundations of Computer Science (FOCS)*, pages 166–175, 2014.
- [31] Rajeev Raman and Satti Srinivasa Rao. Succinct dynamic dictionaries and trees. In *Automata, Languages and Programming*, pages 357–368, Berlin, Heidelberg, 2003. Springer Berlin Heidelberg.
- [32] Omer Reingold, Ron D Rothblum, and Udi Wieder. Pseudorandom graphs in data structures. In *International Colloquium on Automata, Languages, and Programming (ICALP)*, pages 943–954. Springer, 2014.
- [33] Milan Ružić. Uniform deterministic dictionaries. *ACM Trans. Algorithms*, 4(1), March 2008.
- [34] A Siegel. On universal classes of fast high performance hash functions, their time-space tradeoff, and their applications. In *Proceedings of the 30th Annual Symposium on Foundations of Computer Science (FOCS)*, pages 20–25, 1989.
- [35] Alan Siegel. On universal classes of extremely random constant-time hash functions. *SIAM Journal on Computing*, 33(3):505–543, 2004.
- [36] Rajamani Sundar. A lower bound for the dictionary problem under a hashing

model. In *Proceedings 32nd Annual Symposium of Foundations of Computer Science* (FOCS), pages 612–621, 1991.

[37] Mikkel Thorup. Mihai Pătraşcu: Obituary and open problems. *Bulletin of EATCS*, 1(109), 2013.

本文译自 *arXiv:2209.06038v2 [cs.DS]*